התחיל ב:	2/07/2023, 20:46
מצב	הסתיים
הושלם ב-	5/07/2023, 19:04
הזמן שלקח	2 ימים 22 שעות
איחר את המועד	2 ימים 19 שעות

שאלה **1**

לא נענה

ניקוד השאלה: 5.00

נתונה הפונקציה **f** הבאה המקבלת פרמטרים **x, y** חיוביים:

```
def f(x, y):
    if x==0 or y==0:
        return 0

    mn = min(x, y)
    mx = max(x, y)

    count = 0
    while mn < mx:
        mn *= 2
        count += 1

    total = count * (count-1) // 2
    return count
```

מה סיבוכיות זמן הריצה של **f** כפונקציה של **n=max(x,y)**?

רשמו ביטוי (לפי x ו-y) לערך ההחזרה של **f(x,y)**: (שימו לב שמוחזר ערך המשתנה **count** ולא ערך המשתנה **total**.)

התשובה תתקבל עד כדי סטיה של קבוע. לדוגמה, אם התשובה המדוייקת היא **2n+3**, התשובה **2n** תתקבל והתשובה **3n** לא תתקבל.

שאלה **2**

לא נענה

ניקוד השאלה: 5.00

מה סיבוכיות זמן הריצה של **f** כפונקציה של **n**?

```
def f(n):
    count = 0
    for i in range(1, n + 1):
        while i > 1:
            i = i // 2
```

```
            count += 1
    return count
```

שאלה **3**

לא נענה

ניקוד השאלה: 10.00

השלימו במקומות המסומנים את קוד הפונקציה הרקורסיבית **f(n,p=2)** אשר מקבלת שני שלמים **n,p>1**, ומחזירה את מספר הביטויים הכיפליים (factor products) השונים, בהם ניתן להציג את **n**, כאשר כל אחד מהאופרנדים (הכופלים) שלהם הוא מספר שלם גדול או שווה ל **p**.

```
def f(n, p=2):
    if p == n:
        return 1
    elif p > n:
        return 0
    elif n % p > 0:
        return ______(1)______
    return f(______(2)______) +  f(______(3)______)

assert f(7) ==1  # 7
assert f(12)==4  # 2*2*3, 2*6, 3*4, 12
assert f(36)==9  # 2*2*3*3, 2*2*9, 2*3*6, 2*18,
                 # 3*3*4, 3*12, 4*9, 6*6, 36
assert f(36, 6)==2  # 6*6, 36
```

(1)

(2)

(3)

שאלה **4**

לא נענה

ניקוד השאלה: 5.00

השלימו במקומות המסומנים את קוד הגנרטור **my_zip(z1,z2)** אשר מקבל שני איטרטורים ומדמה את פעולת ה-zip עליהם. הגנרטור יפסיק כאשר אחד מהאיטרטורים יפסיק. השתמשו במבנה except ... try מתאים.

```
def my_zip(zit1, zit2):
    while True:
        try:
            ______(1)______
        except ______(2)______:
            ______(3)______

it1 = iter({1:9, 2:9, 'k':9})
it2 = iter(('a', 'b', 'c', 'd'))
gen = my_zip(it1, it2)
assert tuple(gen)==((1, 'a'), (2, 'b'), ('k', 'c'))
```

(1)

(2)

(3)

שאלה **5**

לא נענה

ניקוד השאלה: 5.00

השלימו את הקוד במקום המסומן בגנרטור **tri_pas_line()** אשר מחזיר שורות עוקבות של משולש פסקל בכל זימון של השיטה **next**, כך שכל שורה ארוזה כרשימה.
תזכורת למשולש פסקל:

$$
\begin{array}{c}
1 \\
1 \quad 1 \\
1 \quad 2 \quad 1 \\
1 \quad 3 \quad 3 \quad 1 \\
1 \quad 4 \quad 6 \quad 4 \quad 1 \\
1 \quad 5 \quad 10 \quad 10 \quad 5 \quad 1 \\
1 \quad 6 \quad 15 \quad 20 \quad 15 \quad 6 \quad 1 \\
1 \quad 7 \quad 21 \quad 35 \quad 35 \quad 21 \quad 7 \quad 1
\end{array}
$$

```
def tri_pas_line():
    line = [1]
    while True:
        yield line
        line = ______(1)______

ps = tri_pas_line()
assert next(ps) ==    [1]
assert next(ps) ==   [1, 1]
assert next(ps) ==  [1, 2, 1]
assert next(ps) == [1, 3, 3, 1]
```

(1)

שאלה **6**

לא נענה

ניקוד השאלה: 6.00

(מסגרת השאלה משותפת עם השאלה הבאה)

הניחו שקיימת פונקציה **g(s)** אשר מקבלת מחרוזת **s** שאורכה **z**, ומחזירה מחרוזת שנבנית כך:

```
#g(s) ->  s[0]*z + s[1]*z +
#         s[2]*z + ... + s[z-1]*z
```

הפונקציה **f(s,n)** מקבלת מחרוזת **s** ומספר שלם אי-שלילי **n**. אם **n=0**, הפונקציה **f** תחזיר את המחרוזת **s** עצמה. אחרת, היא מחזירה את התוצאה של הפעלה **n** פעמים של הפונקציה **g** על **s**. דוגמאות:

```
#examples:
assert g('xy')  == 'xxyy'
assert g('x1y') == 'xxx111yyy'
assert g('zz')  == 'zzzz'
```

א. השלימו את הקוד הבא עם שני **פתרונות רקורסיביים שונים אפשריים**.

```
def f(s, n):
    if n == 0:
        return s
    else:
        return _____(1/2)_____

assert f('xy',2)=='xxxxxxxxyyyyyyyy' # (8*'x'+8*'y')
```

על שני הביטויים להיות עם עצי רקורסיה שונים. שימו לב שמדובר בשתי השלמות שונות ובלתי תלויות על אותו קו. הסדר בין הפתרונות לא משנה.

(1)

(2)

שאלה **7**

לא נענה

ניקוד השאלה: 4.00

(מסגרת השאלה משותפת עם השאלה הקודמת)

הניחו שקיימת פונקציה **g(s)** אשר מקבלת מחרוזת **s** שאורכה **z**, ומחזירה מחרוזת שנבנית כך:

```
#g(s) ->  s[0]*z + s[1]*z +
#         s[2]*z + ... + s[z-1]*z
```

הפונקציה **f(s,n)** מקבלת מחרוזת **s** ומספר שלם אי-שלילי **n.** אם **n=0**, הפונקציה **f** תחזיר את המחרוזת **s** עצמה. אחרת, היא מחזירה את התוצאה של הפעלה **n** פעמים של הפונקציה **g** על **s**. דוגמאות:

```
#examples:
assert g('xy')  == 'xxyy'
assert g('x1y') == 'xxx111yyy'
assert g('zz')  == 'zzzz'
```

ב. השלימו את הקוד הבא עם פתרון **לא רקורסיבי**.

```
def f(s, n):
    return ______(3)______

assert f('xy',2)=='xxxxxxxxyyyyyyyy' # (8*'x'+8*'y')
```

שאלה **8**

לא נענה

ניקוד השאלה: 15.00

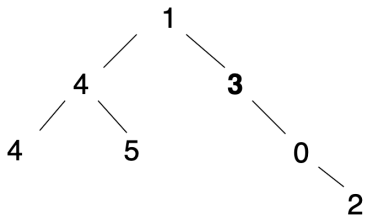
נתונה המחלקה **Tree** הבאה. הניחו שבקדקודים של העצים (**labels**) נמצאים מספרים.

```
class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

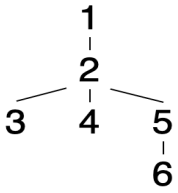
    def is_leaf(self):
        return not self.branches
```

כתבו את הפונקציה **countmax(t)** אשר מקבלת עץ **t** ומחזירה את מספר הקדקודים ב **t** שהערכים שבהם גדולים ממש מהערכים שבכל קדקודי האבות הקדומים שלהם. השורש גדול מקודמיו (באופן ריק) כי אין לו כאלה. דוגמאות:

```
>>> t = Tree(1, [Tree(4, [Tree(4), Tree(5)]), Tree(3, [Tree(0, [Tree(2)])])])
>>> countmax(t)
4  # 1,4,3,5
```



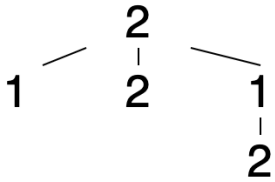

```
>>> t =  Tree(1, [Tree(2, [Tree(3), Tree(4), Tree(5,[Tree(6)])])])
>>> countmax(t)
6  # 1,2,3,4,5,6
```




```
>>> t = Tree(7, [Tree(2, [Tree(3,[Tree(8,[Tree(7)])])])])
>>> countmax(t)
2  # 7,8
```




```
>>> t = Tree(2,[Tree(1), Tree(2), Tree(1,[Tree(2)])])
>>> countmax(t)
1  # 2
```




```
>>> t = Tree(2, [Tree(1, [Tree(5), Tree(7)]), Tree(2, [Tree(0, [Tree(10)])])])
>>> countmax(t)
4  # 2,5,7,10
```

```
>>> t = Tree(-1, [Tree(2), Tree(2)])
>>> countmax(t)
3  # -1,2,2
```

Answer:

(penalty regime: 0 %)

Reset answer

```
ranches=[]):

anches)


s

INE!
```

שאלה **9**

לא נענה

ניקוד השאלה: 15.00

למספר שלם **n** אי-שלילי ביחד עם מספר חיובי של זוגות סוגריים עגולים שעוטפים אותו סימטרית משני צידיו, נקרא ביטוי **מאוזן סוגריים**. אם המספר שווה לעומק הביטוי (מספר הסוגריים שסביבו), הביטוי יקרא גם **תקין**. למשל :

```
(1), ((2)), ((((4)))), ((((((((((10))))))))))...
```

ניתן גם לשלב כמה ביטויים כאלה לביטוי מאוזן אחד גדול יותר:

```
((2)((3))), ((2))((2))
```

כאן, בדוגמה השמאלית, זוג הסוגריים החיצוניים משותפים לביטוי מסביב ל 2 וגם לביטוי שמסביב ל 3 , לכן זהו ביטוי תקין.

דוגמאות לביטויים מאוזנים לא תקינים (בו לא כל המספרים מציינים נכון את עומק הביטוי):

```
(2), ((3)), ((2)(3)), ((3)(2)), ((((12)))), (0), ...
```

דוגמאות לביטויים לא מאוזני סוגריים. (לא תקבלו ביטויים כאלה כקלטים):

```
(), )1(, ((2), (2)), (-1), (a), 0, ...
```

ממשו פונקציה **balance_depth(exp)** אשר מקבלת ביטוי מאוזן ומחזירה **True** אם הביטוי תקין (המספרים שבו מציינים נכון את מספר הסוגריים שסביבם), ו **False** אם הביטוי אינו תקין.

הביטוי המאוזן מתקבל כמחרוזת.

```
>>> exp1 = '((2)((3)))'
>>> balance_depth(exp1)
True
>>> exp2 = '(1)'
>>> balance_depth(exp2)
True
>>> exp3 = '((2)(30))'
>>> balance_depth(exp3)
False
```

הניחו שהקלט הינו ביטוי מאוזן, אין בו תווים אחרים מלבד הסוגריים והמספרים, ואלה אינם מוגבלים לספרה אחת.

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **10**

תקין

ניקוד השאלה: 15.00

השלימו את השיטות החסרות עבור המחלקה **Matrix** הבאה:

```
class Matrix:
    def __init__(self, data):  # הדאטה הוא טופל של טופלים של ערכים
        self.data = data
        # הניחו שהקלט תקין, ושהמימדים חיוביים

    def __str__(self):
        return "\n".join(" ".join(str(x) for x in row) for row in self.data)

    def transpose(self):
        ...  # מחזירה מטריצה חדשה שהיא שחלוף של המטריצה הקיימת
        # (שחלוף = השורות הופכות לעמודות)

    def rotate(self):
        ...  # גנרטור אינסופי שמייצר מטריצות שהן סיבוב בתשעים מעלות (עם כיוון השעון) של המטריצה הקודמת
        # (השורה הראשונה משמאל לימין הופכת להיות העמודה האחרונה מלמעלה למטה)

    def flip_rows(self):
        ...  # מחזירה מטריצה חדשה שהיא שיקוף של המטריצה הקיימת על הציר המאוזן
        # (כלומר, השורה הראשונה הופכת להיות השורה האחרונה)

    def flip_cols(self):
        ...  # מחזירה מטריצה חדשה שהיא שיקוף של המטריצה הקיימת על הציר המאונך
        # (כלומר, העמודה הראשונה הופכת להיות העמודה האחרונה)
```

לדוגמה:

```
>>> m = Matrix(((1, 2, 3),
...             (4, 5, 6),
...             (7, 8, 9)))
>>> print(m)
1 2 3
4 5 6
7 8 9
>>> print(m.transpose())
1 4 7
2 5 8
3 6 9
>>> print(m.flip_rows())
7 8 9
4 5 6
1 2 3
>>> print(m.flip_cols())
3 2 1
6 5 4
9 8 7
>>> a = m.rotate()
>>> print(next(a))
```

```
>>> m = Matrix(((1, 2),
...             (3, 4),
...             (5, 6)))
>>> print(transpose(m))
1 3 5
2 4 6
>>> m = Matrix(((1, 2),))
>>> print(m.transpose())
1
2
>>> m = Matrix(((1, 2, 3),
...             (7, 8, 9)))
>>> print(m.flip_rows())
7 8 9
1 2 3
>>> m = Matrix(((1, 2, 3),))
>>> print(m.flip_rows())
1 2 3
>>> m = Matrix(((1, 2, 3),
...             (7, 8, 9)))
```

הפעלת ארבע פעולות סיבוב:

```
>>> m = Matrix(((1, 2, 3),
...             (4, 5, 6)))
>>> a = m.rotate()
>>> print(next(a))
4 1
5 2
6 3
>>> print(next(a))
6 5 4
3 2 1
>>> print(next(a))
3 6
2 5
1 4
>>> print(next(a))
```

```
7 4 1
8 5 2
9 6 3
>>> print(next(a))
9 8 7
6 5 4
3 2 1
```

```
>>> print(m.flip_cols())
3 2 1
9 8 7
>>> m = Matrix(((1, 2, 3),))
>>> print(m.flip_cols())
3 2 1
```

```
1 2 3
4 5 6
```

Answer:

(penalty regime: 0 %)

Reset answer

```
in(str(x) for x in row) for row in self.data)

INE!
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **11**

לא נענה

ניקוד השאלה: 15.00

הניחו קיום המחלקה **Link** הבאה:

```
class Link:
    empty = ()

    def __init__(self, data, next = empty):
        self.data = data
        self.next = next

    def __repr__(self): return f'Link({repr(self.data)}{f", {repr(self.next)}" if self.next else ""})'
    def __str__(self): return f'({self.data}){f"->{self.next}" if self.next else ""}'
```

במנגנון של list comprehension אפשר לזהות שתי פונקציות "עזר" : אחת מתפקדת כמיפוי, והשניה כפילטר. לדוגמה, בביטוי:

```
>>> lizCom = [k*3 for k in range(10) if k%3==1]
>>> lizCom
[3, 12, 21]
```

הביטוי **k*3** מתפקד כפונקצית מיפוי, והביטוי **if k%3==1** מתפקד כפילטר.

כתבו פונקציה **link_comprehension(lnk, mp, filt)** אשר מקבלת רשימה מקושרת **lnk**, ושתי פונקציות בעלות ארגומנט אחד: **mp** (בתפקיד **map**) ו-**filt** (בתפקיד **filter**). הפונקציה **link_comprehension** תעבור על קדקודי ה **lnk** באמצעות שתי הפונקציות הללו, ותחזיר רשימה מקושרת חדשה אליה יצורפו רק הקדקודים של **lnk** שעבורם **filt** החזירה תוצאה שערך אמת (בוליאני) שלה הוא **True** (כלומר ערך שהפעלת **bool** עליו תחזיר **True**), ואלה יטופלו על ידי הפונקציה **mp** ויכנסו לרשימה החדשה.

אין להשתמש ברשימות או במבני נתונים אחרים (שאינם בגודל קבוע).

דוגמאות:

```
>>> lnk = Link(4, Link(5, Link(7, Link(3, Link(6)))))
>>> link_comprehension(lnk, lambda k: 3*k, lambda k: k % 3 == 1)
Link(12, Link(21))

>>> link_comprehension(lnk, lambda k: k-1/2, lambda k: k > 4 )
Link(4.5, Link(6.5, Link(5.5)))
```

Answer:

(penalty regime: 0 %)

Reset answer

```
xt = empty):


f'Link({repr(self.data)}{f", {repr(self.next)}" if self.next else ""})'
'({self.data}){f"->{self.next}" if self.next else ""}'

INE!

 filt):
```

שאלה **12**

לא נענה

ניקוד השאלה: 15.00

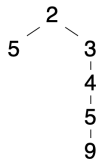
נתונה המחלקה **Tree** הבאה. הניחו שבקדקודים של העצים (**labels**) נמצאים מספרים שלמים.

```
class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)
```

```
    def is_leaf(self):
        return not self.branches
```

כתבו את הפונקציה **consecutive_seq(t)** אשר מקבלת עץ **t** שבקדקדיו מספרים שלמים, ומחזירה את אורך סדרת הקדקדים העוקבים הארוכה ביותר, אשר הערכים שבהם עוקבים והם בסדר עולה.

```
>>> t1 = Tree(2, [Tree(5), Tree(3, [Tree(4 ,[ Tree (5 , [ Tree (9)])])])])
>>> consecutive_seq(t1)
4  # 2->3->4->5
```




```
>>> t2 = Tree(1)
>>> consecutive_seq(t2)
1  # 1
>>> t3 = Tree(1, [Tree(3, [Tree(2)])])
>>> consecutive_seq(t3)
1  # 1 (or 3 or 2)
>>> t4 = Tree(1, [Tree(3, [Tree(4)])])
>>> consecutive_seq(t4)
2  # 3->4
```

Answer:

(penalty regime: 0 %)

Reset answer

```
ranches=[]):

anches)


s

INE!
```

שאלה **13**

תקין

ניקוד השאלה: 15.00

הניחו שקיימות שתי הפונקציות **helper(x,y,n)** ו-**pyramid(n)** כאשר **n** שלם אי שלילי, ו **x,y** הן פונקציות בעלות ארגומנט אחד כל אחת.

```
def helper(x, y, n):
    if n:
        x(n)
        y(n)

def pyramid(n):
    up(n)
    print(n)
    down(n)
```

הפונקציה **pyramid(n)** מקבלת מספר שלם חיובי **n** ומדפיסה שורה אחר שורה את כל התחיליות (prefixes) הלא ריקות של **n**, עד ל-**n** עצמו (כולל), ולאחר מכן תדפיס את אותן תחיליות (prefixes), אבל בסדר יורד. **n** עצמו יודפס רק פעם אחת. דוגמאות:

```
>>> pyramid(1)
1
>>> pyramid(12)
1
12
1
>>> pyramid(3725)
```

```
3
37
372
3725
372
37
3
```

כתבו את שתי הפונקציות החסרות, **up** ו-**down**, כדי שהפונקציה **pyramid** תעבוד נכון.

בשאלה זו בלבד, אין להשתמש בפונקציות עזר מלבד זו שסופקה בשאלה, ואין להשתמש כלל בלולאות.

Answer:

(penalty regime: 0 %)

Reset answer

```
INE!
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **14**

לא נענה

ניקוד השאלה: 1.00

סמנו איזה שאלות בחרתם בחלק ב', השאלות שתסמנו הן אלו שיבדקו **בלבד**.

- שאלה 8 - countmax
- שאלה 9 - balance_depth
- שאלה 10 - Matrix
- שאלה 11 - link_comprehension
- שאלה 12 - consecutive_seq
- שאלה 13 - pyramid

שאלה **15**

לא נענה

לא ניתן ציון

תוכלו לרשום כאן הערות וטקסט חופשי במידת הצורך. **חלק זה אינו נבדק מראש** אלא רק במקרים חריגים ובמקרה של ערעורים.